

dc_steer_v6

Ein DC-gesteuerter Consensus-Optimierer für die TIG-Challenge *neuralnet_optimizer*

Was dieser Optimierer tut, warum er so gebaut ist, wie er sich in The Innovation Game einfügt – und an welchen Stellen er sich sinnvoll weiterentwickeln lässt.

5 Track-Implementierungen

7.194 Zeilen Rust + CUDA

DC-Steering als Kernidee

DIE KERNIDEE

Der Optimierer berechnet in geschlossener Form, um welchen konstanten Betrag der Netzausgang danebenliegt, spielt die Korrektur als kurzen Puls genau vor der Validierung ein – und nimmt sie danach wieder zurück. Bewertet wird das beste gesicherte Modell; genau dieser Punkt sieht die Korrektur.

DER BEFUND

Handwerklich weit über dem Üblichen: ein Kernelstart je Schritt, deterministische Rechenwege, gepaarte Messungen mit t-Werten. Zugleich läuft ein erheblicher Teil der Maschinerie in der ausgelieferten Konfiguration gar nicht – der Vorsprung stammt aus einem kleineren Teil des Programms, als sein Umfang vermuten lässt.

DIE EMPFEHLUNG

Die Regelkreise auf die Epochenachse legen, den Rauschboden messen statt raten, und den Puls vom Versatz auf einen vollständigen Bewertungs-Parametersatz erweitern. Dreizehn Vorschläge in drei Horizonten, jeweils am Code verankert, finden sich in Abschnitt 7.

GEGENSTAND

tig-algorithms/src/
neuralnet_optimizer/dc_steer_v6

REPOSITORY

Erikiss/tig-monorepo
Branch main

STAND

17. September 2026

URHEBER DES ALGORITHMUS

ChervovNikita, 2026

CHALLENGE

neuralnet_optimizer
Tracks n = 4, 7, 10, 14, 18

GRUNDLAGE

Quelltextanalyse
keine eigenen Benchmarkläufe

Dieser Bericht ist eine unabhängige technische Einschätzung auf Basis des Quelltextes im genannten Repository. Er ist keine Veröffentlichung der TIG Foundation und nicht mit dem Urheber des Algorithmus abgestimmt. Alle quantitativen Aussagen sind entweder aus dem Code abgeleitet und mit Fundstelle belegt oder als Angabe des Autors im Quelltext gekennzeichnet; eigene Trainingsläufe waren nicht Teil der Untersuchung.

Inhalt

- 1** Management-Summary
- 2** Kontext: das Spielfeld TIG, die Challenge, und was tatsächlich gemessen wird
 - 2.1** The Innovation Game in einem Absatz
 - 2.2** Die Challenge neuralnet_optimizer
 - 2.3** Was der Code misst – und wo die Beschreibung abweicht
 - 2.4** Der Gestaltungsraum des Innovators
- 3** Aufbau von dc_steer_v6
 - 3.1** Verteilung nach Netztiefe
 - 3.2** Der gemeinsame Kern
 - 3.3** Umfang
 - 3.4** Herkunft: was diese Einreichung beisteuert
- 4** Die fünf Tracks im Einzelnen
 - 4.1** Familie A – Dual-Phase Consensus (T29, T30, T26)
 - 4.2** Familie B – Prometheus (T27, T28)
 - 4.3** Der Rechenkern: ein Start pro Schritt
- 5** DC-Steering – der eigentliche Beitrag
 - 5.1** Ein Versatz, den der Gradient nicht sieht
 - 5.2** Die Herleitung
 - 5.3** Der Puls-Zyklus
 - 5.4** Gain-Liniensuche und der Regressionskoeffizient
 - 5.5** Einordnung: wie neu ist das?
- 6** Bewertung Stärken, Befunde, Zustand der Codebasis
 - 6.1** Was überzeugt
 - 6.2** Ein großer Teil der Maschinerie läuft nicht
 - 6.3** Epochensignal, Schrittauflösung
 - 6.4** Reproduzierbarkeit
 - 6.5** Wie diese Befunde zu lesen sind
 - 6.6** Befunde im Überblick
- 7** Wie man dc_steer_v6 weiterentwickeln könnte dreizehn Vorschläge in drei Horizonten
 - 7.1** Schnell umsetzbar
 - 7.2** Mittelfristig
 - 7.3** Strategisch
 - 7.4** Wege, die ich nicht gehen würde
 - 7.5** Auf einen Blick
- 8** Übertragbarkeit über TIG hinaus
- A** Anhang Dateien, Vorgabewerte, Lizenzen, Methodik

1 Management-Summary

dc_steer_v6 ist kein Optimierer, sondern fünf. Für jede der fünf Netztiefen, die die TIG-Challenge `neuralnet_optimizer` kennt, hält der Algorithmus eine eigene, getrennt abgestimmte Implementierung bereit. Zusammengehalten werden sie von einem gemeinsamen Kern und von einer Idee, die dem Algorithmus den Namen gibt: **DC-Steering**.

1.1 Worum es geht

Die Challenge gibt eine Trainingsschleife vor und lässt genau eine Stellschraube offen: die Funktion, die aus Parametern und Gradienten die Parameteraktualisierung berechnet. Alles andere – Modell, Daten, Batchgröße, Abbruchkriterium – ist fixiert. Der Innovator konkurriert also nicht über Architektur oder Datenaufbereitung, sondern ausschließlich über die Qualität des Optimierungsverfahrens innerhalb eines festen Rechenbudgets.

1.2 Was dc_steer_v6 tut

Die fünf Tracks zerfallen in zwei Familien. Die eine – T29, T30, T26 – berechnet in jedem Schritt mehrere Aktualisierungsrichtungen nebeneinander (Adam, normalisierter Gradient, Vorzeichen-schritt, ein Fisher-vorkonditionierter Schritt) und mischt sie abhängig davon, ob sie sich einig sind; wo sie sich im Vorzeichen widersprechen, übernimmt die robustere Schätzung. Die andere Familie – T27, T28 – baut auf einem Adan/AdaBelief-Kern auf und schaltet je nach Track Muon-Orthogonalisierung, MARS-Varianzreduktion und eine Sophia-artige Krümmungsbegrenzung hinzu.

Beide Familien werden über den Validierungsverlust gesteuert: Lernrate, Mischungsgewichte, Vertrauensgrenzen und Dämpfung hängen davon ab, ob sich der Verlust gerade verbessert, stagniert oder verschlechtert, und wie nah er an einem laufend geschätzten Rauschboden liegt.

1.3 Der Kernbeitrag

DC-STEERING IN EINEM SATZ

Der Algorithmus berechnet in geschlossener Form, um welchen konstanten Betrag der Netzausgang verschoben werden muss, damit er im Mittel auf die Trainingsziele trifft, spielt diese Verschiebung als kurzen Puls genau vor der Validierung ein – und nimmt sie im nächsten Schritt wieder zurück.

Das ist bemerkenswert, weil es eine echte Lücke in der Aufgabenstellung ausnutzt, ohne die Regeln zu verletzen. Die beiden letzten Schichten des Netzes sind eingefroren; ein systematischer Versatz am Ausgang lässt sich über den Gradienten nur mühsam ausgleichen. Der Betrag dieses Versatzes ist aber aus den Trainingszielen trivial berechenbar, und der Weg dorthin führt durch bekannte, eingefrorene Gewichte. Der Algorithmus löst also ein kleines lineares System statt zu warten, bis der Gradientenabstieg dasselbe auf Umwegen erledigt. Bewertet wird nicht das Endmodell, sondern das beste zwischengespeicherte – und genau dieser Prüfpunkt sieht die Korrektur.

1.4 Befund

Handwerklich ist der Code deutlich über dem, was man in einem Benchmark-Beitrag erwartet: ein Kernelstart pro Schritt statt eines pro Tensor, deterministische Festkomma-Reihenfolgen in den kritischen Rechnungen, und eine Arbeitsweise, die auf byte-genaue Vergleichbarkeit

zwischen Varianten ausgelegt ist. Die Kommentare dokumentieren gepaarte Messungen mit t-Werten – das ist eine Sorgfalt, die man selten sieht.

Dem stehen drei Dinge gegenüber. Erstens ist der Umfang hoch: 7.194 Zeilen für eine Aufgabe, deren Referenzlösung mit rund neunzig auskommt, bei erheblicher Dopplung zwischen den Tracks. Zweitens sind die Zahlenwerte, die den Vorsprung ausmachen, nahezu vollständig eingebackten und nicht aus dem Lauf abgeleitet – verschiebt die Challenge ihr Gitter, muss neu abgestimmt werden. Drittens fehlt die Datei, auf die sich der Code für seine Messergebnisse beruft.

1.5 Die drei wichtigsten Weiterentwicklungen

1 Den Rauschboden schätzen statt raten

Der Algorithmus steuert an einer Größe namens `noise_variance` entlang, die er heuristisch aus dem Validierungsverlauf ableitet. Die Aufgabe erlaubt es, denselben Wert sauber zu schätzen. Das macht aus einer abgestimmten Konstante eine gemessene Größe – und nimmt der Abstimmung ihre Zerbrechlichkeit.

2 DC-Steering zu Ende denken

Die Korrektur behandelt bislang nur den konstanten Anteil und nur über eine Schicht. Der gleiche Weg trägt auch eine Maßstabskorrektur – und die ist bei quadratischem Fehler genauso direkt wirksam wie der Versatz.

3 Fünf Tracks auf einen bringen

Die beiden Familien lassen sich als eine parametrisierte Implementierung schreiben. Das halbiert die Codebasis, macht jede künftige Verbesserung fünffach wirksam und beseitigt die Gefahr, dass eine Korrektur nur in vier von fünf Kopien ankommt.

2 Kontext: das Spielfeld

2.1 The Innovation Game in einem Absatz

TIG dreht Proof-of-Work um. Statt Rechenleistung an einer künstlichen Hashaufgabe zu verbrennen, richtet das Protokoll sie auf acht echte Rechenprobleme – Erfüllbarkeit, Tourenplanung, Rucksackproblem, Vektorsuche, Hypergraphpartitionierung, Jobshop-Planung, Energiearbitrage und eben die Optimierung neuronaler Netze. Zwei Rollen greifen ineinander: *Innovatoren* reichen Algorithmen ein, *Benchmarkers* lassen sie laufen und liefern Lösungsbeweise. Weil Benchmarkers ein wirtschaftliches Interesse daran haben, den jeweils effizientesten Algorithmus zu wählen, entsteht aus ihrer Auswahl ein schwer manipulierbares Signal darüber, welcher Beitrag der beste ist – und daran hängt die Belohnung. Der Fachbegriff im Protokoll dafür ist *Optimisable Proof of Work*.

Für den Innovator heißt das: Der Gegner ist nicht die Aufgabe, sondern das Feld. Es reicht nicht, die Schwelle zu reißen; man muss sie deutlicher reißen als alle anderen, und zwar innerhalb eines Rechenbudgets, das das Protokoll *Fuel* nennt. Bemerkenswert daran: Es gibt **keine Zeitschranke** – weder in der Ausführungsumgebung noch im Protokoll findet sich ein Zeitlimit. Gezählt werden Operationen. Auf der CPU geschieht das über eine instrumentierte Übersetzung, die einen Zähler `__fuel_remaining` herunterzählt; auf der GPU wird das Budget vor dem Start direkt in den PTX-Text eingesetzt und mit dem Faktor 20 skaliert, um es grob mit CPU-Kosten vergleichbar zu machen. Wer sein Budget überschreitet, liefert keine Lösung.

Wie ein Algorithmus konkret verdient, steht in `tig-protocol` und ist aufschlussreich für die Bauweise. Jeder Benchmarkers hat eine Obergrenze dafür, wie viele seiner Arbeitspakete je Block zählen – und diese Grenze wird von seiner *schwächsten* Challenge bestimmt. Das erzwingt Breite über alle acht Probleme. Innerhalb einer Challenge werden die Pakete je Track nach absteigender Quality sortiert und in zufälliger Reihenfolge abgearbeitet, bis die Obergrenze erschöpft oder das Track-Kontingent voll ist. Wer knapp hinter dem Feld liegt, qualifiziert sich nicht – nicht, weil er die Schwelle verfehlt hätte, sondern weil andere besser waren.

WARUM DAS DIE BAUWEISE ERKLÄRT

Vieles, was an `dc_steer_v6` zunächst übertrieben wirkt – fünf getrennte Implementierungen, eingebaute Zahlenwerte bis zur dritten Nachkommastelle, ein Kernelstart statt zwölf –, ist die direkte Folge dieser Anreizlage. In einem Wettbewerb, der relative Vorsprünge belohnt und Rechenzeit bestraft, ist Spezialisierung keine Nachlässigkeit, sondern die rationale Antwort.

2.2 Die Challenge `neuralnet_optimizer`

Die Aufgabe trainiert ein mehrschichtiges Perzeptron mit Batch-Normalisierung auf einem synthetischen Regressionsproblem. Die Zielfunktion wird aus 128 zufälligen Fourier-Merkmalen erzeugt – eine Summe von Kosinustermen mit zufälligen Frequenzen und Phasen – und anschließend mit gaußschem Rauschen der Standardabweichung $\sigma = 0,2$ überlagert. Aus 1.450 erzeugten Punkten gehen 1.000 ins Training, 200 in die Validierung, 250 in den Test.

Größe	Wert	Fundstelle
Eingangsdimension	1	<code>mod.rs:136</code>
Ausgangsdimension	2	<code>mod.rs:137</code>
Breite der verborgenen Schichten	256	<code>mod.rs:214</code>
Zahl der verborgenen Schichten	4, 7, 10, 14, 18	<code>Track { n_hidden }</code>

Eingefrorene Schichten	2	mod.rs:219
Batchgröße	128	mod.rs:215
Epochen, höchstens	1000	mod.rs:216
Geduld beim Abbruch	50	mod.rs:217
Fourier-Merkmale K	128	mod.rs:132
Längenmaß der Kernfunktion	0,3	mod.rs:134
Rauschstärke σ	0,2	mod.rs:135

Tabelle 1 — Die festen Größen der Challenge. Fundstellen in `tig-challenges/src/neuralnet_optimizer/mod.rs`.

Zwei dieser Zeilen verdienen besondere Aufmerksamkeit. **Die Schwierigkeit ist kein Zahlenpaar mehr, sondern eine Auswahl.** Eine Aufgabeninstanz wird über `Track { n_hidden }` beschrieben – nichts weiter. Ein Feld `difficulty` existiert im Datenmodell überhaupt nicht mehr; `BenchmarkSettings` führt eine `track_id`. Die ältere Maschinerie für zweidimensionale Schwierigkeitsgitter liegt zwar noch in `tig-utils/src/frontiers.rs`, wird aber von keiner Struktur des aktuellen Modells mehr benutzt.

Es gibt also nicht ein Gitter aus Tiefe und Genauigkeitsanforderung, sondern **fünf diskrete Aufgaben**. Dass `dc_steer_v6` genau fünf Implementierungen hat, ist deshalb keine Übertreibung, sondern eine vollständige Fallunterscheidung – und der übliche Vorwurf, ein Beitrag sei „auf das Schwierigkeitsgitter überangepasst“, greift hier ins Leere: Ein Gitter gibt es nicht.

Und zwei Schichten sind eingefroren. `num_frozen_layers = 2` bedeutet, dass die letzten beiden linearen Schichten – darunter die Ausgabeschicht – ihre zufälligen Anfangswerte behalten. Das Netz muss also durch eine feste, zufällige Projektion hindurch lernen. Genau diese Einschränkung ist die Angriffsfläche, die DC-Steering nutzt (Abschnitt 5).

2.3 Was der Code misst – und wo die Beschreibung abweicht

Bewertet wird das beste während des Laufs gesicherte Modell auf dem Testteil. Die Kennzahl heißt *Quality* und ist eine Festkommazahl mit sechs Nachkommastellen:

```
// tig-challenges/src/neuralnet_optimizer/mod.rs:274-280
let epsilon_star_squared =
    (4.0 / self.dataset.test_size as f32) * sum_sq_diff_true_vs_noisy;
let quality = (epsilon_star_squared as f64 - avg_model_loss_on_test as f64)
    / epsilon_star_squared as f64;
let quality = quality.clamp(-10.0, 10.0) * QUALITY_PRECISION as f64;
```

Quality misst also den *relativen Abstand unterhalb der Schwelle*, in Millionsteln. Wer die Schwelle genau trifft, bekommt null; wer sie um zehn Prozent unterbietet, bekommt 100.000. Das ist kein Bestanden-oder-nicht, sondern eine stetige Skala – und erklärt, warum die Kommentare im Algorithmus um Verbesserungen von wenigen tausend Zähleinheiten ringen.

Rechnet man die Größen aus, ergibt sich ein aufschlussreiches Bild. Der Modellfehler wird als Mittel über alle 500 Ausgabeelemente des Testteils gebildet, die Schwelle dagegen mit dem Faktor $4/N$ über eine Summe, die beide Ausgabedimensionen enthält – also über dieselben 500 Werte, aber geteilt durch 250. Effektiv liegt die Schwelle damit bei $\varepsilon^2 \approx 8\sigma^2 = 0,32$, während selbst ein perfektes Modell den Rauschboden $\sigma^2 = 0,04$ nicht unterbieten kann. Die Annahmegrenze liegt also um den Faktor acht *über* dem Rauschboden – sie ist weit, nicht knapp.

DIE ERREICHBARE HÖCHSTPUNKTZAHL IST EINE EXAKTE ZAHL

Träfe ein Modell die rauschfreie Zielfunktion exakt, wäre sein Testfehler $L = S/500$ mit $S = \sum (y - f)^2$, und die Schwelle wäre $\epsilon^{*2} = S/62,5$. Das Rauschen kürzt sich heraus:

$$Q_{\max} = (1 - 62,5/500) \cdot 10^6 = 875.000$$

Unabhängig davon, wie das Rauschen in einer konkreten Instanz ausfällt. Jede erreichte Quality lässt sich damit unmittelbar als Anteil am Erreichbaren lesen.

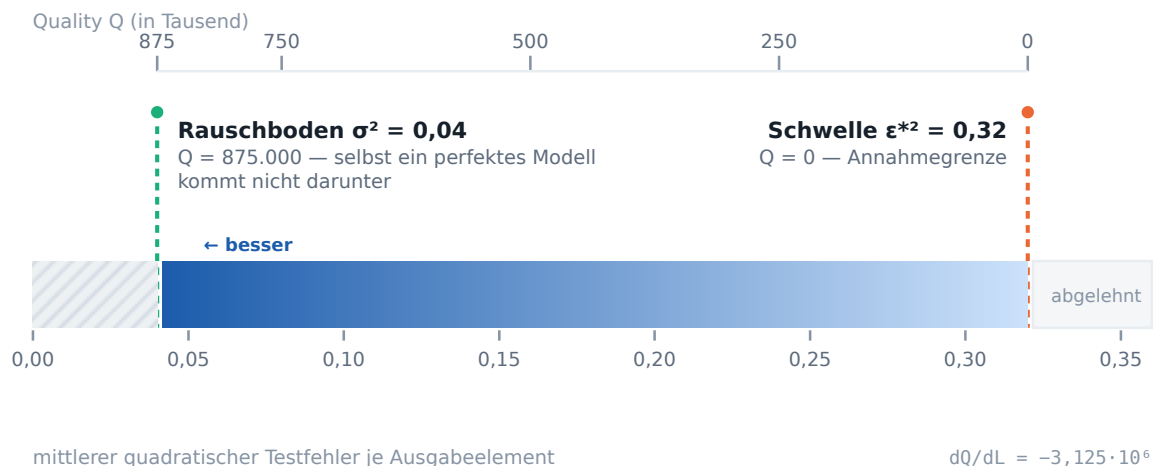


Abbildung 1 — Test-Fehler und Quality sind dieselbe Größe in zwei Einheiten. Die Annahmegrenze ist weit; der Wettbewerb spielt sich im dunklen Teil der Skala ab, nahe am Rauschboden. Die Ableitung dQ/dL ist konstant: eine Verbesserung des Testfehlers um 0,001 ist gut 3.100 Quality-Punkte wert.

Damit lässt sich eine der Messangaben im Quelltext einordnen. Der Kommentar zu Track T26 nennt einen Median-Gewinn von +4.501 Quality-Punkten für eine einzelne Änderung. Das entspricht einer Verbesserung des Testfehlers um rund 0,0014 – gemessen an einem Restabstand zum Rauschboden, der selbst nur wenige Hundertstel beträgt, ist das ein substanzieller Schritt.

BEFUND: BESCHREIBUNG UND CODE LAUFEN AUSEINANDER

Die README der Challenge beschreibt eine ältere Fassung der Aufgabe. Drei Abweichungen sind belegbar:

- **Dimensionen.** Die README nennt acht Eingangs- und eine Ausgangsdimension; der Code arbeitet mit einer Eingangs- und zwei Ausgangsdimensionen.
- **Der zweite Schwierigkeitsparameter fehlt.** Die README beschreibt `accuracy_factor` und `alpha = 4,0 - accuracy_factor/1000`. Im Code ist dieser Parameter *nirgends* zu finden; der Faktor 4,0 steht fest verdrahtet, und eine Aufgabeninstanz kennt nur `n_hidden`.
- **Das Längenmaß.** Die README setzt $\omega \sim N(0, l^{-2})$; der Kernel multipliziert eine standardnormalverteilte Zahl mit $1/l^2 = 11,11$ statt mit $1/l = 3,33$. Das wirksame Längenmaß ist damit 0,09 statt 0,3 – die Zielfunktion schwingt deutlich schneller als dokumentiert.

Wer sich beim Abstimmen auf die README verlässt, optimiert gegen eine Aufgabe, die es nicht mehr gibt.

2.4 Der Gestaltungsraum des Innovators

Die Trainingsschleife liegt vollständig in der Challenge. Der Innovator füllt drei Funktionen aus: eine zum Anlegen des Optimiererzustands, eine optionale, die vor dem Vorwärtsdurchlauf Ersatzparameter vorschlagen darf, und die eigentliche Schrittfunktion, die aus Parametern und Gradienten die Aktualisierungen berechnet.



Der Optimierer sieht `val_loss` — und ist die einzige Stelle, an der der Innovator eingreift.

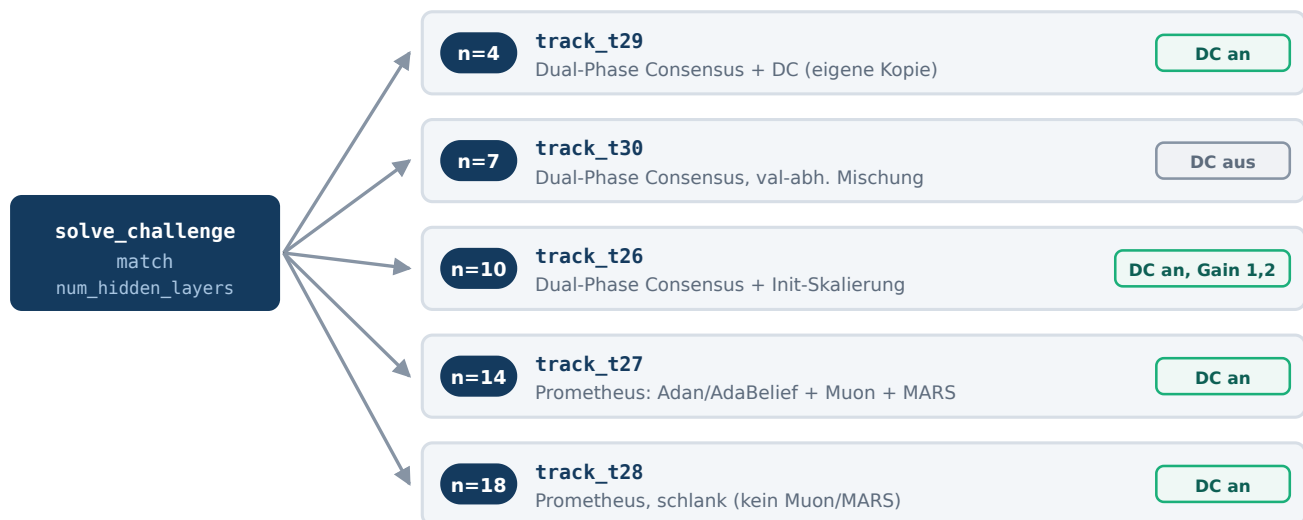
Abbildung 2 — Die feste Trainingsschleife und die beiden Stellen, an denen `dc_steer_v6` eingreift. Der mittlere Einstiegspunkt – der Lookahead-Kanal `optimizer_query_at_params` – wird von allen fünf Tracks ungenutzt gelassen; sie geben `None` zurück.

Zwei Eigenschaften dieser Schnittstelle prägen alles Weitere. Erstens bekommt die Schrittfunktion den **Validierungsverlust** übergeben – eine Rückmeldung über die Generalisierung, die klassische Optimierer nicht haben. `dc_steer_v6` baut darauf seine gesamte Steuerung auf. Zweitens gilt: **bewertet wird das beste gesicherte Modell, nicht das letzte**. Die Trainingsbahn und der bewertete Punkt sind damit nicht dasselbe – und diese Lücke ist der Angelpunkt von DC-Steering.

3 Aufbau von dc_steer_v6

3.1 Verteilung nach Netztiefe

Die Einstiegsfunktion `solve_challenge` tut zwei Dinge: Sie schaltet das Ereignis-Tracking des CUDA-Streams ab, und sie verzweigt anhand von `challenge.num_hidden_layers` in eine von fünf getrennten Implementierungen. Dabei legt sie die für den jeweiligen Track abgestimmten Vorgabewerte auf – überschreibbar, aber vorbelegt.



Jede andere Tiefe → Fehler. Die Challenge kennt genau diese fünf Tracks, deshalb ist die Fallunterscheidung vollständig.

Abbildung 3 — Die Fallunterscheidung in `mod.rs:63–112`. Die Track-Namen folgen der Entwicklungsgeschichte, nicht der Tiefe: `t26` bedient die Tiefe 10, `t29` die Tiefe 4.

Die vorbelegten Werte sind ungleich verteilt. Track T27 (Tiefe 14) bekommt vierzehn eingebackene Größen mit, darunter `ab_blend` 0,55, `ema_decay` 0,9995 und `plateau_patience` 12. Track T30 (Tiefe 7) bekommt eine einzige: `transition_sign` 0,13. Das ist kein Versehen, sondern spiegelt, wie viel Abstimmungsarbeit in welchen Track geflossen ist.

3.2 Der gemeinsame Kern

In `helpers.rs` liegt, was mehrere Tracks teilen: der Zustandstyp `OptimizerState` mit acht Puffern je Parametertensor, der Lernratenplan, die Regeln, nach denen aus dem Validierungsverlauf Mischungsgewichte und Dämpfungen entstehen – und seit dieser Fassung das verallgemeinerte DC-Modul.

Bemerkenswert ist der Zustand selbst. Neben den üblichen ersten und zweiten Momenten führt der Optimierer je Tensor noch den vorigen Gradienten, die vorige Aktualisierung, eine langsame Aktualisierung für den Lookahead-Anteil, eine Fisher-Diagonale und einen Rückstandspuffer für die Vorzeichenquantisierung mit. Acht Puffer in Parametergröße sind, gemessen an Adams zweien, ein erheblicher Speicherpreis – der sich hier lohnt, weil das Netz mit 256 Neuronen Breite klein ist.

STEUERUNG ÜBER DEN VALIDIERUNGSVERLUST

Der interessanteste Teil des gemeinsamen Kerns ist eine Größe namens `noise_variance`. Sie soll den Rauschboden der Aufgabe repräsentieren – jene Verlusthöhe, unterhalb derer kein Fortschritt mehr möglich ist. Der Algorithmus schätzt sie laufend aus dem Validierungsverlauf:

```
// helpers.rs – Zielwert aus dem Verlauf, dann geglättet und gedeckelt
let target_nv = (min_loss / 5.0).min(recent_avg / 8.0);
s.noise_variance = 0.85 * s.noise_variance + 0.15 * target_nv;
s.noise_variance = s.noise_variance.clamp(0.0, 0.05);
```

Die Divisoren 5 und 8 sind frei gewählt, die Obergrenze 0,05 liegt verdächtig nah an der echten Rauschvarianz 0,04. Diese Größe steuert anschließend nahezu alles: ob der Optimierer sich in der „Zone“ nahe dem Boden wähnt und die Lernrate um zwanzig Prozent zurücknimmt, ob er auf den robusten Pfad umschaltet, wie stark er bei Stagnation nachschiebt. **Der Algorithmus hängt damit an einer Größe, die er schätzen könnte, aber heuristisch rät.** Darauf kommt Abschnitt 7 zurück.

3.3 Umfang

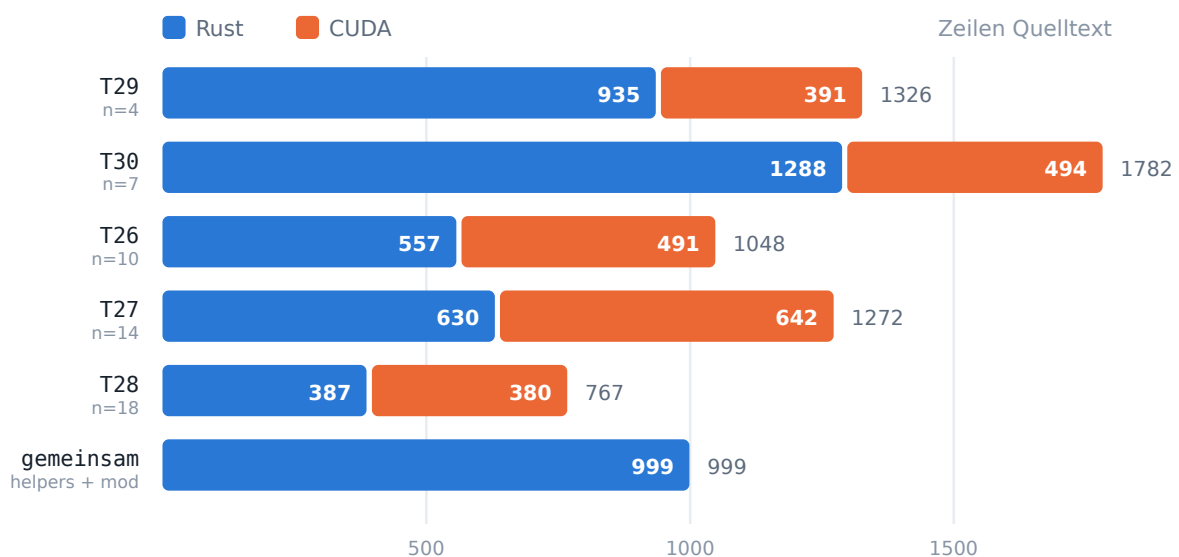


Abbildung 4 — Zeilenumfang je Track, getrennt nach Rust und CUDA. Zum Vergleich: die mitgelieferte Vorlage `template.rs` der Challenge kommt mit rund 90 Zeilen aus.

Der Unterschied zwischen T30 (1.782 Zeilen) und T28 (767 Zeilen) erklärt sich nicht durch die Aufgabe, sondern durch die Bauweise: `track_t30.rs` enthält in einem eigenen Modul eine *vollständige Kopie* der gemeinsamen Hilfsfunktionen, mit track-eigenen Abwandlungen. Dasselbe gilt in abgeschwächter Form für das DC-Modul, das in `track_t29.rs` als handgeschriebener Sonderfall und in `helpers.rs` noch einmal verallgemeinert vorliegt.

BEFUND: DOPPELTE WAHRHEIT

Die DC-Logik existiert zweimal – als handgeschriebene Fassung für Tiefe 4 in `track_t29.rs:162–381` und als verallgemeinertes Modul in `helpers.rs:366–861`, das die vier übrigen Tracks nutzen. Ein Unit-Test stellt sicher, dass die verallgemeinerte Indexrechnung die handgeschriebenen Konstanten reproduziert – aber nur die Indizes, nicht das Verhalten. Jede künftige Änderung am DC-Verfahren muss an beiden Stellen ankommen.

3.4 Herkunft: was diese Einreichung beisteuert

Die Challenge hat elf eingereichte Algorithmen, und sie erzählen eine erkennbare Geschichte. Am Anfang stehen einzelne Dateien von 170 bis 630 Zeilen – `nn_iza`, `adamlaterrrt`, `prometheus_aidda`, `neural_alchemist`, `neural_advanced`. Ab `neural_advanced_v3` beginnt die Aufteilung nach Netztiefe,

und der Umfang springt auf das Dreifache. Von da an wachsen die Beiträge in Stufen: neural_extrem_v3, neural_extrem_v7 – und schließlich dc_steer_v6.

Die Einreichenden wechseln dabei: Rootz baut die Advanced-Linie, NVX die Extrem-Linie, ChervovNikita reicht dc_steer_v6 ein. Das ist bei TIG kein Bruch, sondern das vorgesehene Verhalten: Beiträge stehen unter offenen Lizenzen, damit andere darauf aufbauen. Der Dateivergleich zeigt die Verwandtschaft unmissverständlich – dc_steer_v6 ist eine Fortschreibung von neural_extrem_v7, bis hin zu den Trackdateinamen und dem Text der Hilfeausgabe, die noch immer „Neural Extrem V7“ meldet.

Datei	v7	dc_steer_v6	geänderte Zeilen	Schwerpunkt der Änderung
helpers.rs	324	861	1.185	verallgemeinertes DC-Modul
track_t29.rs	387	935	630	DC von Hand + gebündelter Start
track_t30.rs	1.098	1.288	242	DC-Anbindung
track_t26.rs	406	557	237	DC-Anbindung, init_scale
track_t27.rs	572	630	60	DC-Anbindung
track_t28.rs	329	387	60	DC-Anbindung
mod.rs	120	138	20	Vorgabewerte, dc_gains

Tabelle 2 — Vergleich mit dem Vorgänger. „Geänderte Zeilen“ zählt hinzugefügte und entfernte Zeilen zusammen.

DER BEITRAG IST SCHARF ABGRENZBAR

Eine Suche nach dc_axpy, dc_compute_delta, DcState und dc_gains über neural_extrem_v7 liefert **keinen einzigen Treffer**; dasselbe gilt für den gebündelten Mehrtensor-Start. Beides ist neu in dieser Einreichung. Der Consensus-Kern, die Steuerung über den Validierungsverlust und die Aufteilung nach Tiefe sind dagegen übernommen. *DC-Steering ist also nicht ein Detail dieses Beitrags – es ist der Beitrag.*

4 Die fünf Tracks im Einzelnen

Die fünf Implementierungen sind nicht fünf Varianten desselben Verfahrens, sondern zwei verschiedene Verfahren in fünf Abstimmungen. Was sie eint, ist die Steuerung über den Validierungsverlust und die Art, wie sie auf die GPU gebracht werden.

4.1 Familie A - Dual-Phase Consensus

Tracks T29 (Tiefe 4), T30 (Tiefe 7) und T26 (Tiefe 10). Der Grundgedanke: Statt sich auf eine Aktualisierungsregel festzulegen, berechnet der Kernel drei Kandidaten nebeneinander und mischt sie.

```
// kernels_t29.cu:179-184 – drei Richtungen aus denselben Momenten
const float adam_u = -lr * (m_hat * inv_denom);           // Adam
const float norm_u = -lr * (g_pred * inv_denom);          // normalisierter Gradient
const float sign_u = -lr * copysignf(1.0f, m_hat);        // Vorzeichenschritt (Lion)
const float base_u = blend_adam*adam_u + blend_norm*norm_u + blend_sign*sign_u;
```

Daneben läuft ein zweiter, robusterer Schätzer: eine über die Zeit geglättete Fisher-Diagonale, gespeist mit an ± 6 Standardabweichungen beschnittenen Gradienten, gemischt mit dem reinen Vorzeichenschritt. Die Pointe ist, was danach passiert:

```
// kernels_t29.cu:199-203 – der Konsens
const float agree = (base_u * robust_u >= 0.0f) ? 1.0f : 0.0f;
const float vol   = fminf(sqrtf(fmaxf(v_hat, 0.0f)) * 0.5f, 1.0f);
float mix = (1.0f - agree) * (0.25f + 0.25f*vol) + 0.20f*blend_sign;
float u = (1.0f - mix) * base_u + mix * robust_u;
```

Solange beide Schätzer im Vorzeichen übereinstimmen, bleibt es bei der aggressiven Mischung. Widersprechen sie sich, wandert bis zur Hälfte des Schritts zum robusten Schätzer – und zwar umso mehr, je unruhiger das lokale zweite Moment ist. Das ist eine elegante Formulierung des Grundproblems verrauschter Gradienten: Uneinigkeit zwischen einem momentumbasierten und einem krümmungsbasierten Schätzer ist ein gutes Signal dafür, dass man dem Gradienten gerade nicht trauen sollte.

Danach folgen drei Korrekturen: eine Barzilai-Borwein-artige Skalierung anhand der vorigen Schrittweite, ein *Vertrauensgatter*, das die Schrittlänge in ein Band um den normalisierten Gradientenschritt zwingt (verwandt mit LARS), und ein Lookahead im Aktualisierungsraum – eine langsame geglättete Aktualisierung, die dem schnellen Schritt beigemischt wird. Der Gewichtsabfall wird entkoppelt angewandt, wie bei AdamW.

WAS DIE TRACKS DIESER FAMILIE UNTERSCHIEDET

Bei T30 hängen die Mischungsgewichte an einer Fallunterscheidung über den Trend des Validierungsverlusts: verbessert er sich deutlich, dominiert Adam; stagniert er, verschiebt sich Gewicht zum normalisierten Gradienten; verschlechtert er sich, greift ein Parameter namens `transition_sign` ein, der im Wesentlichen bestimmt, wie viel Vorzeichenschritt in der Übergangsphase beigemischt wird. Genau dieser Wert ist der einzige, den `mod.rs` für diesen Track vorbelegt – mit 0,13 statt der internen 0,15.

T26 ergänzt eine Besonderheit beim Start: Im ersten Schritt addiert der Optimierer (`init_scale - 1`) $\cdot W_0$ auf die Aktualisierung der verborgenen Gewichte – eine nachträgliche Skalierung der Anfangswerte, hier auf den Faktor 3 gesetzt. Da die Initialisierung fest vorgegeben ist, ist das der einzige Weg, sie zu beeinflussen: über die erste Parameteraktualisierung.

DAS SCHRITTBUDGET ALS ABBRUCHMECHANISMUS

Der Lernratenplan läuft nicht in Epochen, sondern in Schritten, und endet hart bei `total_steps`. Bei T30 sind das 1.100 Schritte – bei acht Mini-Batches je Epoche also rund 137 Epochen. Danach ist der Kosinusfaktor exakt null und die Lernrate damit ebenfalls. Die Validierung verbessert sich nicht mehr, und die Geduld des Harness von 50 Epochen beendet den Lauf bei etwa Epoche 188 – knapp ein Fünftel der erlaubten tausend.

`total_steps` ist damit weniger ein Lernratenparameter als ein **Ausstiegskriterium**: Der Algorithmus entscheidet selbst, wann er aufhört, und spart das übrige Rechenbudget. In einem Wettbewerb, der Fuel zählt, ist das ein eigenständiger Hebel.

4.2 Familie B - Prometheus

Tracks T27 (Tiefe 14) und T28 (Tiefe 18). Der Zustandstyp heißt hier `PrometheusState` und verrät die Herkunft: Diese Linie stammt erkennbar aus dem Schwesteralgorithmus `prometheus_aida`. Der Kern ist eine Adan/AdaBelief-Mischung in einem einzigen Kernel:

```
// kernels_t27.cu – der Kern, leicht gekürzt
float m_new = 0.98f*m + 0.02f*g;           // erstes Moment
float v_new = 0.92f*v + 0.08f*g_diff;      // Gradientendifferenz (Adan)
float g_adan = g + 0.92f*g_diff;
n_tgt = ab_blend*g_adan*g_adan + (1-ab_blend)*(g-m_new)*(g-m_new); // AdaBelief-Anteil
float adan_dir = (m_qh + 0.92f*v_new) / (sqrtf(n_new) + eps);
float wd_gate = (adan_dir*theta >= 0.0f) ? 1.0f : 0.0f; // Zerfall nur „nach innen“
float keep = (dir*g > 0.0f) ? 1.0f : 0.25f; // Vorzeichenkonsens
float d_adan = -lr * keep * dir;
```

Zwei Details sind hier interessant. Das `wd_gate` lässt den Gewichtsabfall nur wirken, wenn er in dieselbe Richtung zeigt wie der Schritt – der Parameter wird also nie gegen die Optimierungsrichtung zur Null gezogen. Und `keep` ist wieder ein Konsensgedanke, diesmal in der einfachsten denkbaren Form: Zeigt die berechnete Richtung gegen den rohen Gradienten, wird der Schritt auf ein Viertel gestutzt.

Darüber liegen bei T27 drei zuschaltbare Ebenen, alle über Vorgabewerte gesteuert: **MARS** als vorgeschalteter Kernel, der eine varianzreduzierte Gradientenschätzung bildet; **Muon**, das über eine Newton-Schulz-Iteration die 256×256-Gewichtsmatrizen orthogonalisiert; und eine **Sophia**-artige Variante, die den Wurzelnenner durch eine lineare, beschnittene Form ersetzt. T28 ist dieselbe Architektur ohne diese Ebenen – schlanker, und mit 767 Zeilen der kompakteste Track.

DIE ARBEITSWEISE HINTER DEN SCHALTERN

Jede dieser Ebenen hat einen Vorgabewert, der sie exakt neutralisiert: `mars_gamma = 0` überspringt den MARS-Kernel vollständig, `ab_blend = 1,0` nimmt den AdaBelief-Anteil heraus, `sophia_rho = 0` stellt den Wurzelpfad wieder her. Die Kommentare sprechen von „byte-exakter Parität“ und nennen Prüfwerte. Das ist keine Kosmetik, sondern eine Messmethodik: Man kann jede Erweiterung einzeln zuschalten und weiß sicher, dass der Rest des Programms bitgleich geblieben ist. Wer je versucht hat, den Effekt einer Optimiererkomponente von Gleitkommadrift zu trennen, weiß, wie viel das wert ist.

4.3 Der Rechenkern: ein Start pro Schritt

Die naive Umsetzung startet je Parametertensor einen Kernel – bei Tiefe 7 also 24 Starts je Mini-Batch, mal acht Batches, mal rund 140 Epochen. `dc_steer_v6` fasst das zusammen, nach dem Muster von NVIDIAs `multi_tensor_apply`: Ein einziger Kernel bearbeitet alle Tensoren, jeder Block

findet per Binärsuche über eine Präfixsummentabelle heraus, für welchen Tensor er zuständig ist.

Die Sorgfalt dabei ist bemerkenswert. Die eigentliche Rechnung liegt in `__device__ __forceinline__`-Funktionen, die sowohl vom alten Einzelkernel als auch vom gebündelten aufgerufen werden – *derselbe Text*, damit der schnelle Pfad nicht vom Referenzpfad abdriften kann. Der Kommentar dazu ist unmissverständlich: „DO NOT EDIT WITHOUT RE-RUNNING THE GATE.“

Hinzu kommen kleinere Eingriffe, im Code als P2 bis P6 nummeriert: Statische Teile der Sprungtabelle werden einmalig hochgeladen statt je Schritt; die zurückgegebene Aktualisierungsliste wird auf die Länge gekürzt, die der Harness tatsächlich ausliest; und das Ereignis-Tracking der CUDA-Puffer wird abgeschaltet, weil es in einem Einstrom-Programm nur Kosten verursacht.

NACHGEPRÜFT: DIE UNSAFE-BEGRÜNDUNG TRÄGT

Das Abschalten des Ereignis-Trackings in `mod.rs:57` ist der einzige `unsafe`-Block des Algorithmus. Seine Begründung stützt sich darauf, dass das Programm nur einen einzigen CUDA-Stream benutzt. Eine Suche über `tig-runtime`, `tig-binary` und `tig-challenges` findet genau einen Stream-Bezug – `ctx.fuel_check_stream()` in `tig-runtime/src/main.rs:210` – und keinen einzigen Aufruf von `new_stream()`. Die Voraussetzung hält also; die Sicherheitsargumentation ist unter dieser Bedingung stichhaltig.

5 DC-Steering - der eigentliche Beitrag

Alles Bisherige ist gute Handwerkskunst: bekannte Bausteine, sorgfältig kombiniert und abgestimmt. Der Namensgeber des Algorithmus ist etwas anderes – eine Beobachtung über die Aufgabe selbst, die sich in eine geschlossene Formel übersetzen lässt.

5.1 Ein Versatz, den der Gradient nicht sieht

„DC“ ist hier im nachrichtentechnischen Sinn gemeint: der Gleichanteil, der konstante Sockel eines Signals. Die Trainingsziele haben einen solchen Sockel – ihr Mittelwert ist nicht null. Ein Modell, dessen Ausgang im Mittel daneben liegt, zahlt diesen Versatz im quadratischen Fehler doppelt: Der mittlere quadratische Fehler zerfällt in einen Verzerrungs- und einen Varianzanteil, und ein konstanter Versatz geht als Quadrat direkt in den ersten ein.

Normalerweise erledigt der Gradientenabstieg das nebenbei – die Ausgabeschicht verschiebt ihren Bias, fertig. Hier nicht. **Die letzten beiden linearen Schichten sind eingefroren**; ihr Bias ist unerreichbar. Der Versatz muss durch eine feste, zufällige Projektion hindurch erzeugt werden, aus den Parametern davor. Das kann der Gradientenabstieg zwar, aber langsam und auf Umwegen – er sieht nur den lokalen Anstieg, nicht die algebraische Struktur des Problems.

Der Algorithmus dagegen sieht sie. Der Zielmittelwert ist aus den Trainingsdaten, die der Optimierer ohnehin in der Hand hat, in einer Zeile berechenbar. Der Weg von der letzten trainierbaren BatchNorm-Verschiebung bis zum Netzausgang führt ausschließlich durch bekannte, eingefrorene Gewichte. Also lässt sich direkt ausrechnen, welche Verschiebung nötig ist.

5.2 Die Herleitung

Sei β die Verschiebung der letzten trainierbaren BatchNorm-Schicht, w_3 die darauffolgende eingefrorene lineare Schicht, σ_i die Normierungsgrößen der letzten BatchNorm und w_4 die eingefrorene Ausgabeschicht. Eine kleine Änderung $d\beta$ wandert wie folgt zum Ausgang:

DIE LINEARISIERUNG

$du_i = (w_3 \cdot d\beta)_i \rightarrow d \cdot E[\text{relu}(u_i)] \approx p_i \cdot du_i \rightarrow \text{Division durch } \sigma_i \rightarrow \text{Auslesen über } w_4$

Zusammengefasst ergibt das die Jacobi-Matrix $A[j,k] = \sum_i w_4[j,i] \cdot p_i / \sigma_i \cdot w_3[i,k]$, und gesucht ist δ mit $A \cdot \delta = m$, wobei m der Mittelwert der Trainingsziele je Ausgabedimension ist.

Da A nur zwei Zeilen hat – so viele wie Ausgabedimensionen – und 256 Spalten, ist das System massiv unterbestimmt. Der Code löst es in der Minimalnormlösung über das 2×2 -Gram-System, mit einer Tichonow-Regularisierung von $10^{-6} \cdot \text{spur}$, und deckelt anschließend die Maximumsnorm von δ auf 2.

Der eine nicht beobachtbare Term ist p_i , der Anteil der ReLU-Einheiten, die gerade aktiv sind. Über die Optimiererschnittstelle ist er nicht zugänglich, also wird er auf $\frac{1}{2}$ gesetzt. Die Begründung im Quelltext ist sauber: Weil die Zeilen von w_4 unabhängige Zufallsvektoren sind und die Zeilen von w_3 untereinander kaum korrelieren, wirkt ein falsch angesetztes p fast genau wie ein *Skalar* – die Nebendiagonalen des Fehlerterms sind von der Größenordnung $1/\sqrt{256}$. Ein skalarer Fehler aber lässt sich durch einen einzigen Verstärkungsfaktor auffangen. Genau dafür gibt es die Gain-Liniensuche.

5.3 Der Puls-Zyklus

Die Verschiebung wird nicht dauerhaft eingebaut, sondern als Puls eingespielt und wieder zurückgenommen.

Epoche e — Referenz, kein Puls



Epoche e+1 — Puls

letzter Schritt der Epoche: +g·δ auf die Aktualisierung
der letzten trainierbaren BN-Bias — die Laufstatistik hat den Versatz noch nicht aufgenommen, er kommt am Ausgang an



$$\text{score}(g) = L_1 - L_0$$

Gepulste und ungepulste Epochen wechseln sich ab, deshalb fällt die Drift zwischen Epochen aus der Differenz heraus.

Erster Schritt der Folgepoche: −g·δ — der Puls wird zurückgenommen.

Das Training läuft auf der unveränderten Bahn weiter. Nur der Prüfpunkt, an dem der Harness das beste Modell sichert, sieht die Korrektur.

Nach der Probephase setzt `argmin(score)` den besten Gain dauerhaft; δ wird alle 8 Epochen neu bestimmt.

Abbildung 5 — Ablauf über zwei Epochen. Die Kernstelle ist `helpers.rs:749–836`; die handgeschriebene Fassung für Tiefe 4 steht in `track_t29.rs:237–381`.

Zwei Entscheidungen in diesem Ablauf sind subtil und beide richtig.

Der Zeitpunkt. Der Puls wird im *letzten* Schritt einer Epoche eingespielt, unmittelbar bevor die Validierung läuft. Der Grund steht im Kommentar: So sieht die Validierung die Verschiebung „gegen eine noch vollständig veraltete Laufstatistik“. Hätte die BatchNorm den Versatz bereits in ihre laufenden Mittelwerte aufgenommen, würde sie ihn im Auswertungsmodus wieder herausnormieren – der Puls verpuffte. Er wirkt nur, solange die Statistik ihn noch nicht kennt.

Die Rücknahme. Im ersten Schritt der Folgepoche wird der Puls subtrahiert. Das Training läuft also auf der unveränderten Bahn weiter. Wirksam wird die Korrektur ausschließlich an dem Punkt, an dem der Harness bewertet und das beste Modell sichert. **Der Algorithmus trennt damit die Trainingsbahn von dem Punkt, der bewertet wird** – und nutzt dafür den einzigen Kanal, den die Schnittstelle bietet: die Parameteraktualisierung.

5.4 Gain-Liniensuche und der Regressionskoeffizient

Wie stark der Puls ausfallen soll, wird gemessen statt geraten. In einer Probephase wechseln sich gepulste und ungepulste Epochen ab; bewertet wird die Differenz der Validierungsverluste. Weil die Epochen paarweise verglichen werden, fällt die Drift des laufenden Trainings aus der Differenz heraus – eine gepaarte Messung, wie man sie aus der Versuchsplanung kennt. Nach der Probephase setzt `argmin` den besten Faktor dauerhaft; δ wird alle acht Epochen neu berechnet.

Für die meisten Tracks ist die Kandidatenliste inzwischen einelementig – die Messung ergab, dass eine fünfstufige Leiter denselben Median liefert wie der feste analytische Faktor 1,0, aber mehr Rechenzeit kostet. Eine Ausnahme ist Track T26, und die dortige Begründung ist der interessanteste Kommentar im ganzen Algorithmus:

```
// mod.rs:86-92
// The DC pulse solve linearises the ReLU active fraction at  $p = 0.5$  and assumes the
// pre-pulse eval DC is zero, so it undershoots. The least-squares optimal correction
// is the REGRESSION coefficient  $E[c]/E[t^2] = 1.214$  (not  $1/E[c] = 1.32$  -- the pulse's
// own realisation is stochastic). Measured on this track at  $n=120$  over 8 fresh
// bundles: +4,501 bundle-median ( $t = +2.20$ , 7/8 bundles positive) and 5.0 % FASTER.
// Confirmed dead on  $n_{\text{hidden}}=7$  at the same power (+178,  $t = +0.14$ ), hence per-track.
```

Der Gedanke dahinter: Die Herleitung unterschätzt systematisch, weil sie bei $p = \frac{1}{2}$ linearisiert und unterstellt, der Ausgang habe vor dem Puls keinen Versatz. Die naheliegende Korrektur wäre, durch den mittleren erreichten Anteil zu teilen – $1/E[c] = 1,32$. Das wäre richtig, wenn der Puls genau träfe. Er trifft aber nur im Mittel; seine tatsächliche Wirkung streut. Und wenn die erklärende Größe selbst verrauscht ist, ist der kleinste-Quadrate-optimale Faktor nicht der Kehrwert des Mittelwerts, sondern der *Regressionskoeffizient* $E[c]/E[t^2]$ – der wegen des Rauschanteils im Nenner kleiner ausfällt: 1,214 statt 1,32.

DAS IST STATISTISCH RICHTIG GEDACHT

Der Unterschied zwischen $1/E[c]$ und $E[c]/E[t^2]$ ist genau die klassische Abschwächung durch Fehler in der erklärenden Variablen. Wer den Kehrwert nimmt, überkorrigiert und zahlt den Fehler quadratisch. Der Kommentar dokumentiert zudem Stichprobenumfang, t-Wert, Streuung über acht unabhängige Bündel und den negativen Befund auf einem anderen Track. Das ist eine Sorgfalt, die man in Benchmark-Beiträgen selten findet – und der Grund, warum die Zahl track-spezifisch eingebackten und nicht global gesetzt wurde.

5.5 Einordnung: wie neu ist das?

Jeder Baustein für sich ist bekannt. Ein Ausgabe-Bias an den Zielmittelwert anzupassen, ist Standard. Eine Verstärkung per Liniensuche zu bestimmen, ist Standard. Gepaarte Vergleiche sind Standard. Auch die Idee, an eingefrorenen Schichten vorbei analytisch zu rechnen, findet sich in der Literatur zu Random-Feature-Modellen.

Neu ist die *Kombination im Kontext dieser Schnittstelle*. Der Algorithmus nutzt drei Eigenheiten der Aufgabe gleichzeitig aus: dass die letzten Schichten eingefroren und damit bekannt sind; dass BatchNorm im Auswertungsmodus eine veraltete Statistik verwendet und ein frischer Versatz deshalb durchschlägt; und dass der Harness nicht das Endmodell, sondern das beste Zwischenmodell bewertet. Keine dieser Eigenheiten ist ein Fehler der Challenge – zusammen ergeben sie aber eine Nische, die eine rein gradientenbasierte Methode nicht erreicht.

Ehrlicherweise gehört dazu die Einschränkung: Das ist zu einem guten Teil *Aufgabenausnutzung*, nicht allgemeiner Optimiererfortschritt. Welcher Anteil sich übertragen lässt, behandelt Abschnitt 8.

6 Bewertung

6.1 Was überzeugt

Die Kernidee trägt

DC-Steering ist eine echte Einsicht über die Aufgabe, sauber hergeleitet, korrekt umgesetzt und mit einer statistisch richtig gedachten Korrektur versehen. Das ist mehr als Parametersuche.

Die Messmethodik ist vorbildlich

Gepaarte Vergleiche, mehrere unabhängige Bündel, t-Werte, dokumentierte Negativbefunde, Vorgabewerte, die Erweiterungen byte-exakt neutralisieren. Wer so arbeitet, kann Effekte von Gleitkommadrift trennen – die meisten können das nicht.

Der Rechenkern ist ernsthaft optimiert

Ein Kernelstart je Schritt statt eines je Tensor, einmalig hochgeladene statische Tabellen, gekürzte Rückgabelisten, abgeschaltetes Ereignis-Tracking – und die eigentliche Rechnung liegt in gemeinsamen `__forceinline__`-Rümpfen, damit der schnelle Pfad nicht vom Referenzpfad abweichen kann.

Die Determinismus-Disziplin an den richtigen Stellen

Die DC-Rechnung läuft vollständig in f64 in fester Reihenfolge, explizit als deterministisch dokumentiert. Für ein Protokoll, das Lösungen nachprüft, ist das genau die richtige Sorge.

6.2 Der wichtigste Befund: ein großer Teil der Maschinerie läuft nicht

Liest man den Code als Beschreibung dessen, was im Benchmark tatsächlich passiert, klafft eine Lücke. Eine ganze Reihe von Mechanismen ist in der ausgelieferten Konfiguration unerreichbar oder wirkungslos. Die folgenden Punkte habe ich einzeln am Quelltext nachgeprüft.

VON DREIZEHN CUDA-KERNELS IN T27 LAUFEN ZWEI

`kernels_t27.cu` definiert dreizehn Kernels. Vier werden nirgends geladen: die unfusionierte Altfassung `adabelief_update_kernel_t27`, die beiden Consensus-Kernels `sign_ef_consensus_kernel_14` und `dual_consensus_fisher_kernel_14` sowie `grokfast_ema_kernel`. Sechs weitere – die Newton-Schulz-Kette für Muon – werden nur geladen, wenn `muon_lr_scale > 0` gilt; der Vorgabewert ist 0, und `mod.rs` setzt ihn nicht. `mars_correction_kernel` läuft nur bei `mars_gamma > 0`; `mod.rs:108` backt ausdrücklich 0.0. Auch `sophia_rho` steht auf 0.

WAS DAS BEDEUTET

Muon, MARS und Sophia – die drei Bausteine, die den Track am modernsten aussehen lassen – sind im ausgelieferten Zustand **alle abgeschaltet**. Ausgeführt werden `adabelief_fused_kernel_t27` und `dc_axpy_t27`. Die Bausteine sind über Hyperparameter erreichbar und insofern nicht wertlos; als Beschreibung dessen, was den Benchmarkvorsprung erzeugt, sind sie irreführend.

DIE DÄMPFUNG DER AUSGABESCHICHT KANN NICHT WIRKEN

`output_layer_damping` taucht in allen Tracks auf, mit sorgfältig abgestimmten Werten zwischen 0,77 und 0,86. Die Bedingung in der Schrittfunktion von T29 lautet `i + 2 >= last` mit `last = gradients.len() = 26`, während die Schleife nur über `0..20` läuft – der Zweig ist unerreichbar. In der Initialisierung ist er erreichbar, trifft dort aber die Indizes 24 und 25, also die Laufstatistiken der letzten BatchNorm, die nie aktualisiert werden.

Der tiefere Grund ist einfacher: **Die Ausgabeschicht ist von der Challenge eingefroren.** Sie bekommt überhaupt keine Aktualisierung. Eine Dämpfung ihrer Lernrate kann per Konstruktion nichts bewirken. Derselbe Befund gilt sinngemäß für T30, T26 und T27.

DER GEWICHTSABFALL IN T26 IST KEINER

Der schnelle Pfad von T26 rechnet:

```
// kernels_t26.cu:228
const float adam_update = -lr * (m_hat * inv_denom + weight_decay * g_pred);
```

Der Zerfallsterm multipliziert den *Gradienten*, nicht das Gewicht. Das ist weder klassischer L2-Zerfall noch der entkoppelte Zerfall von AdamW – es ist ein zusätzlicher kleiner SGD-Anteil, der nichts reguliert. Der robuste Pfad derselben Datei macht es zwei Zeilen weiter richtig (`adj_update -= wd_lr * w`), und T29 macht es durchgehend richtig (`final_u -= (lr * weight_decay) * params[idx]`). Innerhalb eines Tracks weichen die beiden Pfade also im Verhalten voneinander ab.

EINGEBACKENE WERTE OHNE ABNEHMER

`mod.rs:90` setzt für T26 `init_scale = 3.0` – eine dreifache Skalierung der Anfangsgewichte, auf den ersten Blick ein kräftiger Eingriff. Die zugehörige Bedingung prüft `s.step_count == 0`, nachdem der Zähler bereits erhöht wurde. Der Kommentar im Code benennt das selbst: `track_t26.rs:370` „*this branch is structurally unreachable and the kernel is never launched ... Kept verbatim for parity*“. Das ist also bekannt und bewusst stehen gelassen – der eingebackene Wert 3,0 suggeriert dennoch eine Wirkung, die es nicht gibt. Ähnlich liegt es bei fünf der vierzehn für T27 vorbelegten Größen, für die der Track keinen Leser hat.

6.3 Ein systematischer Fehlgriff: Epochensignal, Schrittauflösung

Dieser Punkt betrifft alle fünf Tracks und ist der gewichtigste. Der Harness übergibt an `optimizer_step` nicht den aktuellen, sondern den **vorigen** Validierungsverlust – und zwar unverändert für alle acht Mini-Batch-Schritte einer Epoche:

```
// tig-challenges/src/neuralnet_optimizer/mod.rs – gekürzt
let param_updates = optimizer_step(/* ... */, prev_validation_loss, /* ... */);
// ... erst nach allen Batches der Epoche: ...
prev_validation_loss = Some(avg_val_loss); // mod.rs:569
```

Der Optimierer wertet dieses Signal aber je Schritt aus. Die Differenz `improvement = prev - curr` ist deshalb in sieben von acht Schritten *exakt null*. Alles, was daran hängt – der Stagnationszähler, die Trenderkennung, die Steigungsglättung, die Wahl der Mischungsgewichte – arbeitet mit einer Auflösung, die das Signal nicht hergibt. Ein Stagnationszähler, der bei 25 eine Lernraterhöhung auslöst, erreicht diesen Stand nach gut drei Epochen, nicht nach 25.

FOLGE

Die Regelkreise sind nicht falsch programmiert, aber gegen die falsche Zeitachse kalibriert. Da die Schwellenwerte empirisch abgestimmt wurden, „funktionieren“ sie – sie tun nur etwas anderes, als ihre Namen und Kommentare nahelegen. Das erklärt zugleich, warum so viele Konstanten so spezifisch sind: Sie kompensieren eine Verzerrung, die man auch beseitigen könnte.

6.4 Reproduzierbarkeit

Zwei Stellen brechen die Wiederholbarkeit, und beide sitzen in T30 und T26.

Erstens summiert `kernels_t30.cu:28` Blockergebnisse per `atomicAdd` auf einen Gleitkommawert. Gleitkommaaddition ist nicht assoziativ, und die Reihenfolge, in der Blöcke fertig werden, bestimmt die Hardware. Derselbe Lauf auf derselben Karte kann damit unterschiedliche Werte liefern – und dieser Wert steuert die Lernrate.

Zweitens hängt die Blockzahl je Tensor von `prop.multiProcessorCount` ab, und die Kernels reduzieren *je Block*. Ergebnisse sind damit von der GPU abhängig. Für ein Protokoll, dessen Prüfung auf der Wiederholbarkeit eines Laufs beruht, ist das die ernsteste der hier genannten Schwächen.

6.5 Wie diese Befunde zu lesen sind

Es wäre falsch, daraus auf Nachlässigkeit zu schließen. Fast alle toten Pfade sind Rückstände einer *Arbeitsweise*: Es wird eine Erweiterung eingebaut, mit einem neutralisierenden Vorgabewert versehen, gemessen – und wenn sie nichts bringt, bleibt sie ausgeschaltet stehen, statt entfernt zu werden. Der Kommentar „*swap#3 DEAD*“ in `track_t27.rs:74` sagt genau das. Aus Sicht des Wettbewerbs ist das rational: Entfernen kostet Zeit und riskiert, dass sich etwas anderes ändert.

Der Preis ist, dass der Code seine eigene Wirkung nicht mehr korrekt beschreibt. **Der tatsächliche Wettbewerbsvorsprung stammt aus einem deutlich kleineren Teil des Programms, als sein Umfang vermuten lässt** – aus dem Consensus-Kern, der Steuerung über den Validierungsverlust, dem Schrittbudget als Ausstiegskriterium und dem DC-Puls. Das ist eine gute Nachricht für die Weiterentwicklung: Der Teil, der zählt, ist überschaubar.

6.6 Befunde im Überblick

Gewicht	Befund	Fundstelle
hoch	Validierungsverlust ist epochenkonstant und um eine Epoche verzögert; alle trendbasierten Regelkreise arbeiten in Schrittauflösung	<code>tig-challenges/.../mod.rs:509</code> , <code>569 helpers.rs:169–262</code>
hoch	Gleitkomma- <code>atomicAdd</code> in der Lernratensteuerung ⇒ Lauf-zu-Lauf-Abweichung	<code>kernels_t30.cu:28</code>
hoch	Blockzahl hängt von <code>multiProcessorCount</code> ab, Reduktion erfolgt je Block ⇒ GPU-abhängige Ergebnisse	<code>track_t26.rs:137</code> <code>kernels_t26.cu:175–201</code>
mittel	Gewichtsabfall im schnellen Pfad von T26 wirkt auf den Gradienten statt auf das Gewicht; robuster Pfad derselben Datei macht es anders	<code>kernels_t26.cu:228</code> vs. <code>:110</code>
mittel	Muon, MARS und Sophia sind in der ausgelieferten Konfiguration abgeschaltet; 11 von 13 Kernels in T27 laufen nicht	<code>track_t27.rs:596–616</code> <code>mod.rs:108</code>
mittel	<code>output_layer_damping</code> unerreichbar bzw. wirkungslos – die Ausgabeschicht ist eingefroren	<code>track_t29.rs:739</code> <code>challenge num_frozen_layers = 2</code>
mittel	DC-Logik liegt doppelt vor: handgeschrieben für Tiefe 4, verallgemeinert für die übrigen vier Tracks	<code>track_t29.rs:162–381</code> <code>helpers.rs:366–861</code>
mittel	Die im Code viermal zitierte Ergebnisdatei <code>RESULTS.md</code> liegt dem Beitrag nicht bei	<code>track_t26.rs:25</code> , <code>t27:18</code> , <code>t28:17</code> , <code>t30:389</code>
gering	Vorgabewerte ohne Leser: <code>init_scale</code> (dokumentiert unerreichbar), fünf der vierzehn T27-Werte	<code>mod.rs:90</code> , <code>:103–107</code> <code>track_t26.rs:373</code>
gering	Gleitender Mittelwert teilt fest durch 10, obwohl die Bedingung schon ab 6 Einträgen greift – anfänglich zu niedriger Schätzwert	<code>helpers.rs:193–195</code>

gering	Bias-Korrektur nutzt $t = \text{step_count} + 1$, obwohl der Zähler bereits erhöht wurde; zudem ist β_1 nicht konstant, wie die Formel unterstellt	helpers.rs:170 track_t29.rs:675
gering	Genau ein Unit-Test im gesamten Beitrag, und er prüft nur die Indexrechnung des DC-Moduls	helpers.rs:840–859

Tabelle 3 — Befunde, jeweils am Quelltext nachgeprüft. „Gewicht“ bezieht sich auf die Folgen für Korrektheit und Nachprüfbarkeit, nicht auf den Aufwand einer Behebung.

7 Wie man dc_steer_v6 weiterentwickeln könnte

Die Vorschläge sind nach Horizont geordnet und jeweils an einer Stelle im Code verankert. Wo ich den erwarteten Nutzen nicht beziffern kann, sage ich das. Alle bleiben innerhalb der Regeln – es wird nur Optimiererlogik geändert; wo ein Vorschlag an die Grenze geht, steht es dabei.

7.1 Schnell umsetzbar

1 — DIE REGELKREISE AUF DIE EPOCHENACHSE LEGEN S HOHE WIRKUNG

Der Validierungsverlust ändert sich einmal je Epoche. Alles, was aus seinem Verlauf Trends, Stagnation oder Steigungen ableitet, gehört genau einmal je Epoche ausgewertet – erkennbar an `s.last_epoch != epoch`, wie es `track_t27.rs` für seinen Plateauzähler bereits richtig macht. Betroffen sind `update_state_from_val_loss` und `compute_global_damp` in `helpers.rs` sowie die eingebettete Kopie in `track_t30.rs`.

Wichtig: Das ist *keine* reine Aufräumarbeit, sondern eine Verhaltensänderung. Die Schwellenwerte wurden gegen die verzerrte Achse abgestimmt und müssen mitwandern – ein Stagnationszähler von 25 Schritten entspricht gut drei Epochen. Der Gewinn ist, dass die Parameter danach bedeuten, was sie heißen, und dass die Regelung das Signal mit voller statt mit einem Achtel der wirksamen Auflösung nutzt.

2 — DEN NICHTDETERMINISMUS BESEITIGEN S HOHE WIRKUNG

Zwei gezielte Eingriffe: Die Gleitkomma-atomicAdd-Reduktion in `kernels_t30.cu:28` durch eine zweistufige Reduktion mit fester Reihenfolge ersetzen, und die Blockzahl nicht mehr von `prop.multiProcessorCount` abhängen lassen, sondern fest aus der Tensorlänge ableiten. Beides kostet kaum Rechenzeit und macht Läufe zwischen Karten und zwischen Wiederholungen vergleichbar – was für ein Protokoll, das Lösungen nachprüft, mehr wert ist als ein weiterer Bruchteil Quality.

3 — DEN REGRESSIONSKOEFFIZIENTEN AUF ALLEN TRACKS MESSEN S MITTLERE WIRKUNG

Für T26 ist `dc_gains = [1.2]` belegt und mit +4.501 Median bei $t = 2,20$ begründet. Für T7 wurde derselbe Wert gemessen und als wirkungslos verworfen. Für **T27, T28 und T29 liegt keine Messung vor** – dort steht weiterhin der analytische Wert 1,0, obwohl das Argument für die Unterschätzung tiefenunabhängig ist. Die Messapparatur existiert bereits: die Gain-Leiter über `dc_gains` mit gepaarter Bewertung. Das ist der billigste noch offene Hebel im ganzen Algorithmus.

4 — DEN GEWICHTSABFALL IN T26 GERADEZIEHEN S KLEINE WIRKUNG

Eine Zeile: `weight_decay * g_pred` durch einen entkoppelten Term – `lr*wd*w` ersetzen, wie es der robuste Pfad derselben Datei und T29 durchgehend tun. Ob das die Quality verbessert, ist offen – die Konstante 0,0020 wurde gegen das falsche Verhalten abgestimmt. Aber danach stimmen Name und Wirkung überein, und die Konstante lässt sich sinnvoll neu suchen.

5 — AUFRÄUMEN, WAS NACHWEISLICH NICHT LÄUFT S KLEINE WIRKUNG

Die vier nie geladenen Kernels in `kernels_t27.cu`, die Vorgabewerte ohne Leser und die unerreichbaren Zweige entfernen – und die zugehörigen Zustandspuffer gleich mit. Verhaltensneutral per Konstruktion, weil die Pfade nicht laufen. Der Nutzen ist nicht Geschwindigkeit,

sondern dass der nächste Leser – auch der Autor in einem Jahr – dem Code wieder glauben kann. Für Muon und MARS gilt eine Ausnahme: Sie sind über Hyperparameter erreichbar und sollten als solche dokumentiert, nicht gelöscht werden.

6 — RESULTS.MD BEILEGEN S

Vier Dateien berufen sich auf eine Ergebnisdatei, die dem Beitrag nicht beiliegt. Sie ist die Belegkette für jede eingebaute Konstante. Ohne sie ist die sorgfältigste Messmethodik von außen nicht nachvollziehbar – und genau das ist im TIG-Kontext, wo andere Innovatoren auf einem Beitrag aufbauen sollen, der Unterschied zwischen einem Ergebnis und einem Beweis.

7.2 Mittelfristig

7 — DEN RAUSCHBODEN SCHÄTZEN STATT RATEN M HOHE WIRKUNG

Dies ist der Vorschlag mit dem besten Verhältnis von Aufwand zu Wirkung. `noise_variance` steuert praktisch jede Entscheidung im Algorithmus und wird heute als `min(min_loss/5, recent_avg/8)` geschätzt – mit frei gewählten Divisoren und einer Deckelung bei 0,05, die verdächtig nah am wahren Wert 0,04 liegt. Die Folge ist ein selbstbezüglicher Regelkreis: Weil sich `noise_variance` auf ein Achtel des laufenden Mittels einpendelt, ist die Bedingung für die „Präzisionszone“ im eingeschwungenen Zustand fast immer erfüllt. Ein Mechanismus, der ein seltenes Fenster markieren sollte, ist zum Dauerzustand geworden.

Dabei ist die Größe *messbar*, und zwar mit Bordmitteln. Die Eingabe ist eindimensional und gleichverteilt auf $[-1, 1]$, die Zielfunktion ist bandbegrenzt glatt. Damit greift der klassische differenzbasierte Varianzschätzer: Man sortiert die 1.000 Trainingspunkte nach ihrer Eingabe und bildet

DIFFERENZBASIERTER SCHÄTZER DES RAUSCHBODENS

$$\sigma^2 = 1 / (2(n-1)) \cdot \sum_i (y_{i+1} - y_i)^2$$

Der systematische Fehler ist die halbe mittlere quadrierte Funktionsänderung zwischen benachbarten Punkten. Mit dem *wirksamen* Längenmaß 0,09 (siehe Abschnitt 2.3) ist $\text{Var}(f') = l^{-4} \approx 123$; bei 1.000 Punkten auf einem Intervall der Länge 2 ist der mittlere quadrierte Abstand $8 \cdot 10^{-6}$, der Bias also $\approx 5 \cdot 10^{-4}$ gegenüber $\sigma^2 = 0,04$ – **gut ein Prozent**. Die statistische Streuung liegt bei rund vier Prozent, über beide Ausgabedimensionen gemittelt. Wer genauer will, nimmt einen Differenzschätzer zweiter Ordnung; der drückt den Bias um eine weitere Größenordnung. Der Schätzer braucht nur Daten, die das Modell ohnehin sieht, und läuft einmal beim Start.

Damit wird aus einer abgestimmten Konstante eine gemessene Größe. Alle Schwellen, die heute als Vielfache von `noise_variance` formuliert sind, bekommen eine physikalische Bedeutung: Ein Verlust von $1,5 \cdot \sigma^2$ heißt dann wirklich „das Modell erklärt alles bis auf fünfzig Prozent Überschuss über dem Rauschen“. Und – der eigentliche Gewinn – die Abstimmung wird robust gegen Änderungen an der Challenge, weil sie an einer gemessenen statt an einer geratenen Größe hängt.

8 — DC-STEERING UM DIE MASSSTABSKORREKTUR ERWEITERN M HOHE WIRKUNG

Der Puls korrigiert bislang den Versatz. Die kleinste-Quadrat-optimale Nachkalibrierung einer Vorhersage hat aber zwei Freiheitsgrade: $\hat{y} \mapsto a \cdot \hat{y} + b$. Der Versatz b ist erledigt; der Maßstab a nicht. Bei einem Modell, das – wie hier durch die eingefrorene Zufallsprojektion erzwungen – systematisch zur Mitte hin schrumpft, ist gerade der Maßstabsfehler der größere Posten.

Der Weg dorthin ist derselbe: Die *Skalenparameter* der letzten trainierbaren BatchNorm wirken multiplikativ und liegen im selben Aktualisierungsvektor wie die Verschiebungen. Die erforderliche Kovarianz zwischen Vorhersage und Ziel ist auf dem Trainingsteil berechenbar, ohne zusätzliche Vorwärtsdurchläufe, wenn man die Ausgaben einer Epoche mitführt. Die vorhandene Puls-Mechanik – einspielen, validieren, zurücknehmen, per gepaarter Messung bewerten – lässt sich unverändert weiterverwenden.

9 — DIE FÜNF TRACKS AUF EINE PARAMETRIERTE IMPLEMENTIERUNG BRINGEN L

MITTLERE WIRKUNG

Zwei Verfahren, fünf Abstimmungen – das sind zwei Implementierungen plus eine Tabelle, nicht fünf Dateien. Konkret: die eingebettete Helper-Kopie in `track_t30.rs` auflösen, die handgeschriebene DC-Fassung in `track_t29.rs` durch das verallgemeinerte Modul ersetzen (die Indexgleichheit ist bereits getestet), T27 und T28 zu einer Datei mit Schaltern zusammenführen. Der Nutzen ist nicht ästhetisch: Jede der obigen Verbesserungen wirkt danach auf allen Tracks statt auf einem, und die Gefahr, dass eine Korrektur in vier von fünf Kopien ankommt, verschwindet.

10 — EINE REPRODUZIERBARE MESSUMGEBUNG BEILEGEN M MITTLERE WIRKUNG

Die Messmethodik des Autors – gepaarte Nonces, Bündelmediane, t-Werte, Paritäts-Prüfwerte – existiert offensichtlich, aber nur außerhalb des Beitrags. Ein beiliegendes Skript, das eine Konfiguration über n Nonces gegen eine Referenz laufen lässt und Median, t-Wert und Laufzeit ausgibt, würde aus privatem Handwerk ein nachnutzbares Werkzeug machen. Dasselbe gilt für Tests: Der eine vorhandene Unit-Test prüft Indexarithmetik; testbar wären ohne GPU auch die Lernratenpläne, die Mischungsgewichte und der DC-Solver gegen eine Referenzimplementierung.

7.3 Strategisch

11 — DEN PULS VERALLGEMEINERN: EIN EIGENER PARAMETERSATZ FÜR DIE BEWERTUNG L HOHE WIRKUNG

Dies ist die konsequente Fortführung der Kernidee und der interessanteste offene Zug. DC-Steering nutzt aus, dass der Harness *das beste validierte Modell* sichert, nicht das zuletzt trainierte – und dass der Optimierer über den Aktualisierungskanal bestimmen kann, welche Parameter zum Validierungszeitpunkt im Netz stehen. Bisher wird dieser Kanal für einen 256-elementigen Versatzvektor genutzt. Er trägt aber *jede* Parameteränderung.

DER VORSCHLAG

Der Optimierer führt einen gleitenden Mittelwert θ_{avg} der Parameter mit. Im letzten Schritt einer Epoche spielt er die Differenz $\theta_{\text{avg}} - \theta$ als Puls ein; die Validierung sieht damit den gemittelten Parametersatz. Ist er besser, sichert der Harness ihn. Im ersten Schritt der Folgepoche wird der Puls zurückgenommen, und das Training läuft auf der ungemittelten Bahn weiter.

Das ist Gewichtsmittelung – ein seit Langem etabliertes, verlässliches Mittel gegen die Streuung später Trainingsphasen – die hier *ohne* die übliche Nebenwirkung zu haben ist, dass man zwischen gemitteltem und laufendem Modell wählen müsste. Man bekommt beides: eine ungestörte Trainingsbahn und einen gemittelten Bewertungspunkt. Der Aufwand ist ein zusätzlicher Puffer je Tensor und zwei zusätzliche Kernelstarts je Epoche.

DER HAKEN, DEN MAN MESSEN MUSS

Die Rücknahme ist additiv – der Harness kennt nur $\theta \leftarrow \theta + \Delta\theta$. In f32 gilt aber $(\theta + d) - d \neq \theta$. Beim heutigen DC-Puls betrifft das 256 Werte mit kleiner Amplitude; bei vollen Parametersätzen über zweihundert Epochen kann sich die Abweichung aufsummieren. Vor dem Einbau gehört das quantifiziert – etwa, indem man den Puls nur auf die größten Schichten anwendet oder nur in der Spätphase, in der die Mittelung ohnehin am meisten bringt.

Ein zweiter, bereits bestehender Haken: Die Rücknahme erfolgt einen Mini-Batch zu spät. Der Harness rechnet Vorwärts- und Rückwärtsdurchlauf, *bevor* er `optimizer_step` aufruft – der erste Batch der Folgeepoche sieht den Puls also noch. Beim kleinen DC-Versatz fällt das kaum ins Gewicht; bei einem vollen Parametersatz ist es ein echter Störfaktor, den man über den ungenutzten Kanal `optimizer_query_at_params` sauber lösen könnte.

12 — DIE EINGEBACKENEN KONSTANTEN DURCH EINEN SELBSTEINSTELLUNGSLAUF ERSETZEN L MITTLERE WIRKUNG

Heute bricht `solve_challenge` bei jeder anderen Tiefe als 4, 7, 10, 14 oder 18 mit einem Fehler ab. Das ist derzeit vollständig – die Challenge kennt nur diese fünf –, aber es ist eine Wette darauf, dass sich das nicht ändert. Eine tiefenabhängige Interpolation der wichtigsten Größen plus ein generischer Rückfallpfad kostet wenig und nimmt dem Beitrag seine Sprödigkeit.

Weiter gedacht: Die ersten zwanzig bis dreißig Epochen sind ohnehin Aufwärmphase. Man könnte in dieser Zeit zwei oder drei Konfigurationen gegeneinander laufen lassen und nach dem Muster eines mehrarmigen Banditen entscheiden – mit derselben gepaarten Bewertungslogik, die die Gain-Leiter schon benutzt. Aus einem eingebackenen Abstimmungsergebnis würde eine im Lauf getroffene Entscheidung.

13 — DAS TOTAL_STEPS-BUDGET SELBST BESTIMMEN M MITTLERE WIRKUNG

Die Schrittzahl ist heute je Track eine feste Zahl zwischen 1.110 und 2.850 – sie bestimmt sowohl den Lernratenplan als auch faktisch den Abbruchzeitpunkt. Beide Rollen ließen sich trennen: Der Plan behält sein nominelles Budget, und der Ausstieg wird an eine gemessene Bedingung gehängt – etwa daran, dass der Validierungsverlust den geschätzten Rauschboden um weniger als einen festen Faktor überschreitet und sich über k Epochen nicht mehr signifikant bewegt. Das ist genau die Art Entscheidung, für die der geschätzte Rauschboden aus Vorschlag 7 die Grundlage liefert.

7.4 Wege, die ich nicht gehen würde

Weitere Optimiererbausteine zuschalten

Muon, MARS, Sophia, QHAdam und der EMA-Nesterov-Aufsatz sind bereits eingebaut – und stehen alle auf neutral. Das ist kein Zufall, sondern das Ergebnis von Messungen. Ein sechster Baustein wird daran nichts ändern; das Problem ist nicht die Auswahl der Aktualisierungsregel.

Die eingebackenen Konstanten feiner absuchen

Der Ertrag sinkt erkennbar – die dokumentierten Gewinne liegen im Bereich weniger tausend Quality-Punkte bei t -Werten um 2. Jede weitere Nachkommastelle erhöht die Zerbrechlichkeit, ohne die Ursache zu verbessern. Erst die Regelkreise geradeziehen, dann neu abstimmen.

Länger trainieren

Dass die Tracks bei einem Fünftel der erlaubten Epochen aussteigen, ist kein Versehen, sondern eine Entscheidung gegen das Fuel-Budget. Mehr Epochen kosten unmittelbar und bringen nahe am Rauschboden fast nichts.

Die rauschfreien Zielwerte heranziehen

Der Datensatz stellt über `dataset.train_targets_true_f()` auch die rauschfreien Ziele bereit, und technisch könnte der Optimierer sie lesen. Damit ließe sich der Rauschboden exakt bestimmen – und weit mehr. Das wäre allerdings Information, über die kein reales Training verfügt, und der Geist der Aufgabe ist eindeutig. Vorschlag 7 erreicht praktisch dasselbe mit Daten, die das Modell ohnehin sieht; das ist der saubere Weg.

7.5 Auf einen Blick

#	Maßnahme	Aufwand	Wirkung	Ansatzpunkt
1	Regelkreise auf die Epochenachse legen	S	hoch	helpers.rs:169–336, track_t30.rs
2	Nichtdeterminismus beseitigen	S	hoch	kernels_t30.cu:28, track_t26.rs:137
3	Regressionskoeffizient je Track messen	S	mittel	mod.rs dc_gains, helpers.rs:807
4	Gewichtsabfall in T26 entkoppeln	S	gering	kernels_t26.cu:228
5	Nachweislich toten Code entfernen	S	gering	kernels_t27.cu, mod.rs:90
6	RESULTS.md beilegen	S	mittel	Beitragsverzeichnis
7	Rauschboden differenzbasiert schätzen	M	hoch	helpers.rs:192–199 + solve()
8	DC um die Maßstabskorrektur erweitern	M	hoch	helpers.rs::dc
9	Fünf Tracks auf eine Implementierung	L	mittel	track_t26...t30.rs
10	Messumgebung und Tests beilegen	M	mittel	Beitragsverzeichnis
11	Gemittelter Parametersatz für die Bewertung	L	hoch	helpers.rs::dc::step
12	Selbsteinstellung statt eingebackener Werte	L	mittel	mod.rs:63–112
13	Ausstieg an eine gemessene Bedingung hängen	M	mittel	helpers.rs spectral_phase_lr

Tabelle 4 — Aufwand: **S** unter einer Woche, **M** ein bis zwei Monate, **L** darüber. „Wirkung“ bezieht sich auf Quality *und* Nachprüfbarkeit; Nummer 2 und 6 verbessern vor allem Letztere.

8 Übertragbarkeit über TIG hinaus

Die ehrliche Frage zum Schluss: Was von alldem taugt außerhalb dieses Benchmarks? Die Antwort zerfällt sauber in zwei Teile – und der übertragbare Teil ist kleiner, aber interessanter, als der Umfang des Codes vermuten lässt.

8.1 Was sich überträgt

Der Konsensgedanke

Zwei Schätzer der Aktualisierungsrichtung zu bilden – einen momentumgetriebenen, einen krümmungsgetriebenen – und bei Vorzeichenwiderspruch dem robusteren mehr Gewicht zu geben, ist kein Kniff für diese Aufgabe, sondern eine allgemeine Antwort auf verrauschte Gradienten. In T27 steht die einfachste Form davon: $\text{keep} = (\text{dir} \cdot \text{g} > 0) ? 1 : 0,25$ – der Schritt wird gestutzt, wenn er dem rohen Gradienten widerspricht. Das ähnelt auffällig dem, was in der jüngeren Literatur als *cautious* Optimierer beschrieben wird, wo die Maske hart auf null geht. Ein Vergleich beider Varianten auf üblichen Vergleichsmessungen wäre eine kleine, gut abgegrenzte Studie.

Nachkalibrierung am Bewertungspunkt

Der allgemeine Kern von DC-Steering hat mit TIG nichts zu tun: *Bevor man ein Modell bewertet oder sichert, lohnt sich eine affine Nachkalibrierung seiner Ausgabe.* Zwei Parameter, auf einer Validierungsmenge geschätzt, kosten nichts und verringern den quadratischen Fehler garantiert nicht. In der Prognoserechnung ist das Standard; in der tiefen Regression wird es überraschend selten gemacht. Dass hier der Umweg über BatchNorm-Verschiebungen nötig war, ist eine Eigenheit der Schnittstelle – der Gedanke dahinter ist allgemein.

Die Asymmetrie zwischen Trainings- und Auswertungsmodus

Der meines Erachtens interessanteste übertragbare Befund. BatchNorm normiert im Auswertungsmodus mit einer *veralteten* Statistik. Eine frische Parameteränderung schlägt deshalb an den Ausgang durch, während dieselbe Änderung nach ein paar Epochen von der nachgezogenen Statistik weitgehend wieder herausgerechnet würde. Wer Parameter nachträglich verändert – beim Beschneiden von Netzen, beim Quantisieren, beim Verschmelzen zweier Modelle –, trifft auf genau dieses Phänomen, meist als unerklärliche Abweichung zwischen den Moden. Hier wird es zum ersten Mal, soweit ich sehe, absichtlich *genutzt*. Das verdient eine saubere Untersuchung.

Die Arbeitsweise

Jede Erweiterung mit einem Vorgabewert zu versehen, der sie byte-exakt neutralisiert, und jede Änderung gegen gepaarte Läufe zu messen – das ist ein Verfahren, das jede Gruppe übernehmen kann, die an Optimierern arbeitet. Der übliche Zustand in diesem Feld ist, dass Verbesserungen von Gleitkommadrift und Startwertglück nicht zu trennen sind. Hier sind sie es.

8.2 Was nicht überträgt

Die eingebackenen Konstanten sind Abstimmungsergebnisse für genau diese fünf Aufgaben. Die analytische DC-Lösung setzt voraus, dass die letzten Schichten eingefroren *und damit bekannt* sind – ein realistisches Training hat diese Struktur nicht. Und die Zonenlogik rund um `noise_variance` hängt an einem Rauschniveau, das hier bekannt und konstant ist.

Man sollte das nicht als Kritik lesen. Der Beitrag ist für einen Wettbewerb geschrieben, der Spezialisierung belohnt, und erfüllt diesen Zweck. Nur: Wer ihn als allgemeinen Optimierer einsetzen will, nimmt im Wesentlichen den Consensus-Kern mit – und der ist rund ein Zehntel

des Codes.

8.3 Drei prüfbare Hypothesen

Wenn jemand den übertragbaren Teil belegen wollte, wären das die Experimente:

1. **Konsens schlägt Vorsicht.** Der weiche Übergang zwischen aggressivem und robustem Schätzer, gesteuert über Vorzeichenübereinstimmung und lokale Streuung, erreicht bei gleichem Rechenbudget einen niedrigeren Testfehler als eine harte Maske und als reines AdamW. Zu prüfen auf mehreren Aufgaben mit unterschiedlich verrauschten Gradienten; die Vorhersage ist, dass der Vorsprung mit dem Rauschniveau wächst.
2. **Nachkalibrierung ist ein freies Mittagessen.** Eine affine Nachkalibrierung der Ausgabe auf der Validierungsmenge, angewandt bei der Wahl des Prüfpunkts, senkt den Testfehler auf Standard-Regressionsaufgaben messbar und ohne Trainingskosten. Die Vorhersage ist, dass der Gewinn mit der Stärke der Regularisierung steigt, weil Schrumpfung zur Mitte genau den Maßstabsfehler erzeugt, den die Kalibrierung behebt.
3. **Die Statistik-Verzögerung ist quantifizierbar.** Wie viel der Wirkung einer Parameteränderung im Auswertungsmodus überlebt, ist eine fallende Funktion der Zahl der Trainingsschritte zwischen Änderung und Auswertung, mit der Trägheit der Laufstatistik als Zeitkonstante. Das ist direkt messbar – und hätte unmittelbare Folgen für alle Verfahren, die Netze nachträglich verändern.

Keine dieser Hypothesen braucht TIG. Alle drei stammen aus dem Code.

A Anhang

A.1 Dateien und Umfang

Eingereicht am 7. August 2026 durch die Adresse 0x8e671c8f...8cd2; im Repository als Commit 99f931b „*Player 0x8e671c... submitted code neuralnet_optimizer/dc_steer_v6*“ mit 14 Dateien und 7.221 hinzugefügten Zeilen. Die Übersetzung in eine Bibliothek erfolgt anschließend durch TIG selbst, mit einer angepassten LLVM-Fassung, die Fuel-Zählung und Laufzeitsignatur einfügt.

Datei	Zeilen	Rolle
mod.rs	138	Einstieg, Verteilung nach Tiefe, eingebackene Vorgabewerte
helpers.rs	861	Gemeinsamer Zustand, Lernratenplan, Steuerung, DC-Modul, ein Unit-Test
track_t29.rs	935	Tiefe 4 – Consensus + eigene DC-Fassung
track_t30.rs	1.288	Tiefe 7 – Consensus mit eingebetteter Helper-Kopie
track_t26.rs	557	Tiefe 10 – Consensus + Init-Skalierung
track_t27.rs	630	Tiefe 14 – Prometheus mit Muon/MARS/Sophia (abgeschaltet)
track_t28.rs	387	Tiefe 18 – Prometheus, schlank
kernels_t26...t30.cu	2.398	CUDA-Kernels, je Track eine Datei
README.md	23	Einreichungsangaben und Lizenzhinweis
Summe	7.194	4.796 Rust + 2.398 CUDA

A.2 Die eingebackenen Vorgabewerte

Was mod.rs je Track vorbelegt, bevor der Track seine eigenen Vorgaben anwendet. Nutzerseitig übergebene Hyperparameter haben immer Vorrang. Grau hinterlegte Zeilen sind Werte, für die der jeweilige Track keinen Leser hat.

Track	Tiefe	Vorgabewerte aus mod.rs
t29	4	total_steps 1900 · warmup_steps 16 · beta2 0,999 · weight_decay 0,015 · bn_layer_boost 1,0 · spectral_boost 1,25 · nv_hi_lo 5,5
t30	7	transition_sign 0,13
t26	10	total_steps 2850 · warmup_steps 200 · bn_layer_boost 0,95 · noise_variance 0,025 · spectral_boost 0,95 · init_scale 3,0 · dc_gains [1,2]
t27	14	total_steps 1950 · warmup_steps 200 · bn_layer_boost 0,95 · noise_variance 0,025 · spectral_boost 0,95 · ab_blend 0,55 · ema_decay 0,9995 · mars_gamma 0,0 · min_lr_scale 0,5 · t_max_epochs 850 · nesterov_beta 0,8 · mars_clip_scale 1,0 · plateau_patience 12 · hidden_bias_scale 1,5
t28	18	total_steps 1110 · ghw_scale 1,0

Tabelle 5 — Auffällig ist die Ungleichverteilung: T30 bekommt eine einzige Vorgabe, T27 vierzehn, von denen fünf nirgends gelesen werden.

A.3 Lizenzlage

Die README des Beitrags nennt sechs Lizenzen gleichzeitig. Das ist kein Fehler, sondern das Lizenzmodell von TIG: Verschiedene Nutzungen greifen auf verschiedene Lizenzen zu.

Für Innovatoren, die darauf aufbauen wollen

Die *TIG Innovator Outbound Game License*. Genau dieser Weg wurde hier beschritten – dc_steer_v6 baut auf neural_extrem_v7 eines anderen Einreichenden auf.

Für Benchmarker

Die *TIG Benchmarker Outbound Game License* regelt den Einsatz im Proof-of-Work-Betrieb.

Für kommerzielle Nutzung außerhalb des Spiels

Die *TIG Commercial License*. Wer Teile dieses Codes in ein Produkt übernehmen möchte – etwa den Consensus-Kern –, braucht sie. Die *TIG Open Data License* deckt den offenen Forschungsteil ab.

Praktisch heißt das für einen Leser dieses Berichts: Die *Ideen* aus Abschnitt 8 sind frei verwendbar; das *Übernehmen des Quelltextes* in ein kommerzielles Produkt ist es nicht. Die Lizenztexte liegen unter docs/licenses/ im Repository.

A.4 Zur Methodik dieses Berichts

Grundlage ist ausschließlich der Quelltext im genannten Repository, Stand `main` am 17. September 2026. Es wurden **keine eigenen Trainingsläufe** durchgeführt; die Challenge benötigt eine CUDA-fähige GPU, die hier nicht zur Verfügung stand.

Daraus folgt eine klare Grenze, die für alle Aussagen dieses Berichts gilt:

- **Belegt** sind alle Aussagen über Struktur, Rechenwege, erreichbare und unerreichbare Zweige sowie über die Formeln der Challenge. Jede davon ist am Code nachgeprüft und mit Datei und Zeile belegt.
- **Angabe des Autors** sind alle Zahlen zu gemessenen Quality-Gewinnen – die +4.501 bei $t = 2,20$ etwa stammen aus einem Codekommentar und sind hier nicht nachgemessen worden. Die zugehörige Belegdatei `RESULTS.md` liegt dem Beitrag nicht bei.
- **Abgeleitet** sind die quantitativen Aussagen in Abschnitt 2.3 und in Vorschlag 7: Sie folgen aus den Konstanten der Challenge und sind im Text mitgerechnet, aber nicht empirisch überprüft.
- **Einschätzung** ist die Gewichtung in Tabelle 3 und die Reihenfolge in Abschnitt 7. Ob eine der vorgeschlagenen Änderungen die Quality tatsächlich verbessert, kann nur eine gepaarte Messung zeigen – mit genau der Apparatur, deren Fehlen Vorschlag 10 anspricht.

FUNDSTELLEN

Sofern nicht anders angegeben, beziehen sich Dateinamen ohne Pfad auf `tig-algorithms/src/neuralnet_optimizer/dc_steer_v6/`. Die Challenge liegt unter `tig-challenges/src/neuralnet_optimizer/`, die Ausführungsumgebung unter `tig-runtime/`. Zeilennummern beziehen sich auf den oben genannten Stand und können sich mit künftigen Änderungen verschieben.